

Bueno, buenos días a todos.

1ª Parte: INTRODUCCIÓN

Hoy vamos a contaros qué es un ataque Sybil y qué relación tiene con la programación concurrente y distribuida.

En primer lugar, hay que entender por qué un ataque Sybil es importante. Por suerte, junto con esta asignatura también estamos viendo redes de ordenadores, así que entendemos lo que es una red y la importancia que tiene que una red sea segura. Ahora bien, a alguien que le interese más el ámbito de las redes le sonarán conceptos como blockchain o Tor.

La blockchain de Bitcoin, por ejemplo, funciona sobre una red peer to peer de nodos que intercambian información sin una autoridad central, y Tor es una red distribuida orientada a la privacidad total y al anonimato, a través de tor tu te puedes conectar a la red de onion donde se gestiona toda la deepweb.

Estamos hablando de sistemas muy grandes, muy robustos y muy importantes, y precisamente por eso resulta relevante estudiar ataques que intentan aprovechar puntos débiles en este tipo de redes.

2ª Parte: EXPLICACIÓN ATAQUE

Un ataque Sybil no consiste necesariamente en “tumbar” toda una blockchain o toda la red Tor de golpe, sino en introducir muchas identidades falsas dentro de una red P2P para ganar presencia, influencia y capacidad de manipulación sobre su funcionamiento.

Ahí está la importancia de este ataque: si una red no puede distinguir bien entre nodos legítimos e identidades falsas, el atacante puede empezar a controlar parte de la estructura interna de esa red.

Dicho esto, vamos a explicar internamente cómo funciona:

Ya hemos mencionado que este ataque se centra en redes peer to peer, es decir, redes en las que distintos nodos se comunican entre sí sin depender de un servidor central. En primer lugar, el atacante necesita acceso a la red o, como mínimo, la posibilidad de enviar solicitudes de entrada a los nodos de esa red.

Una vez puede interactuar con ella, su objetivo deja de ser entrar con una sola identidad y pasa a ser aparentar que existen muchos nodos distintos controlados por él. Al final, una red peer to peer está construida por nodos que se van conectando entre sí y manteniendo relaciones de vecindad, así que si un atacante consigue meter muchas identidades falsas dentro de esa estructura, empieza a ocupar espacio que debería pertenecer a nodos honestos.

Entonces, una vez que el atacante ya está en posición de actuar, genera muchas IDs distintas que aparentan ser nodos diferentes, aunque en realidad todas pertenezcan al mismo origen.

Esas identidades falsas no son personas ni dispositivos reales independientes, sino identidades simuladas por el atacante para hacerse pasar por múltiples participantes dentro de la red.

Después, en nuestro caso, se crean hilos y cada hilo se asocia al intento de conexión de una de esas identidades falsas. Esto es justamente donde entra la programación concurrente, porque el ataque no se lanza de forma secuencial, sino en paralelo, haciendo que muchas identidades intenten entrar casi al mismo tiempo.

A continuación, cada una de esas identidades envía peticiones a la red.

En nuestro proyecto, esas peticiones son mensajes JOIN, que son solicitudes normales de entrada a la red y que, por tanto, intentan hacerse pasar por peticiones legítimas. La comunicación se hace sobre TCP, pero es importante decirlo bien: TCP no decide si el nodo es seguro o no; TCP simplemente transporta la conexión, y luego es nuestro programa el que comprueba si acepta o rechaza esa petición. En concreto, cada nodo revisa cosas como la IP de origen, la reputación de la identidad, la prueba de trabajo si está activada, y también si todavía tiene espacio libre en su tabla de vecinos.

Por eso no se puede asegurar siempre que el ataque funcione. Si la red detecta demasiadas conexiones desde la misma IP, o si la identidad no cumple las condiciones necesarias, la petición se rechaza y ese nodo falso no entra. Pero si esas comprobaciones no frenan la solicitud, entonces la identidad falsa es aceptada y pasa a formar parte de la red como si fuera un vecino legítimo.

Y aquí es donde empieza realmente el problema. Si el atacante consigue que entren muchas de esas identidades falsas, empieza a llenar las tablas de vecinos de los nodos legítimos. Eso significa que cada vez hay menos hueco para nodos honestos y más presencia del atacante dentro de la red. Dicho de forma simple, el atacante no destruye la red desde fuera, sino que se mete dentro de ella con muchas identidades falsas hasta ganar peso en su funcionamiento interno.

En nuestro código, la red funciona mediante nodos que escuchan conexiones, aceptan mensajes y mantienen listas de vecinos. El atacante lo que intenta es colar muchas identidades falsas dentro de esas listas de vecinos para ir ganando presencia y capacidad de influencia. Así, cuantas más identidades falsas consigue meter, mayor control potencial tiene sobre la red, porque aparenta ser muchos participantes distintos cuando en realidad solo hay un atacante detrás. Y esa es precisamente la lógica de un ataque Sybil: aparentar descentralización cuando en realidad una parte importante de la red está controlada por una sola entidad.

3ª Parte: NUESTRO ATAQUE

Entonces, una vez explicado lo que es un ataque sibyl y lo que implica vamos a pasar a explicar como hemos hecho nosotros un ataque sibyl, bueno no lo hemos hecho, lo hemos simulado, vale. Para eso tenemos el siguiente video

En primer lugar, Lo que hemos hecho aquí es simular un ataque Sybil dentro de una red P2P creada por nosotros. La red tiene un nodo que actúa como punto inicial de registro, y varios nodos legítimos que se comunican por TCP y mantienen relaciones de vecinos.

Una vez que la red está montada y estabilizada, entra en juego el atacante. Ese atacante genera 20 identidades falsas y lanza conexiones en paralelo para intentar colarse en distintos nodos como si fueran participantes reales de la red.

Cada nodo receptor decide si acepta o rechaza la petición de entrada aplicando límites por IP, control de reputación y, si se activa, el proof of work. Es decir, estamos dejando por sentado las cualidades o la seguridad de la red dando al TCP las reglas que tiene que pasar un dispositivo para acceder a la red.

En nuestra ejecución concreta, 11 identidades fueron bloqueadas, pero 9 consiguieron entrar, así que el ataque no fue totalmente frenado y la red quedó parcialmente comprometida.

4ª Parte: RESULTADOS

En nuestra simulación trabajamos con 5 nodos legítimos y 20 identidades Sybil. De esas 20 identidades falsas, 11 fueron bloqueadas y 9 consiguieron entrar en la red. Eso significa que las defensas funcionaron solo de forma parcial, porque redujeron el ataque, pero no lo detuvieron por completo. Por tanto, la red quedó parcialmente comprometida, que es justo lo que queríamos observar con la práctica.

5ª Parte: CIERRE

Este trabajo se relaciona directamente con la asignatura porque combina programación concurrente y sistemas distribuidos. Por un lado, usamos concurrencia mediante hilos, tanto en los nodos legítimos como en el atacante, para gestionar conexiones y lanzar ataques en paralelo. Por otro lado, trabajamos con una red distribuida de nodos que se comunican por TCP mediante intercambio de mensajes. En resumen, el proyecto nos ha permitido aplicar en un caso real conceptos de concurrencia, comunicación distribuida, coordinación entre nodos y seguridad en red.